SpendWise — Integration Specification

Connect real API data, authentication, budgets and analytics to the UI

What this document controls

Use after the backend is generated and its actual routes and schemas are known. This phase removes mock-only behavior and connects the frontend to real APIs.

COPY THE FULL PROMPT BELOW INTO ANTIGRAVITY

Use this prompt as one message. Let Antigravity inspect the repository, make the requested changes, run checks, and report what it actually completed. Do not skip the checkpoints.

Integrate the existing SpendWise Next.js frontend with the actual AWS backend. Inspect the current frontend and backend code and their response schemas first. Do not invent endpoints or silently change contracts.

```
AUTH
- Integrate Cognito using a maintained compatible library or documented hosted UI flow.
- Configure public User Pool ID, client ID and region through public environment variables where needed. These values are not
- Attach a valid access token to authenticated API calls.
- Handle sign-in, sign-out, expired sessions and unauthenticated routes.
- Do not use localStorage alone as proof of authentication.
- Do not implement a fake login that grants access to backend data.

API CLIENT
Create or update a typed `lib/api.ts` with:
- Base URL from NEXT_PUBLIC_API_BASE_URL.
- Shared request helper that adds auth token, JSON headers and handles errors.
- Typed functions for list/create/get/update/delete transactions.
- Typed functions for list/create/update/delete budgets.
- Typed function for analytics summary.
- Optional receipt presign/upload function only if backend supports it.
- Safe handling for network errors, 401, 403, 404, 409 and 5xx responses.
- No secrets in browser environment variables.

FEATURE CONNECTION
- Replace mock transactions with real API requests.
- Create/edit/delete transactions and refresh the relevant dashboard totals after success.
- Apply filters through the API when supported; otherwise filter only the currently fetched data and clearly implement pagina
- Connect budget forms and progress cards to DynamoDB-backed API data.
- Ensure analytics use the selected month and actual records.
- Ensure “Can I Afford This?” uses the current month's actual budget and expenses and explains its calculation.
- Keep user data isolated; never cache one user's data for another user.
- Show loading, empty, validation, error and success states.
- Do not show success unless the API operation succeeded.
- Do not treat an optimistic UI update as persisted until the request succeeds.

ENVIRONMENT
Provide `.env.example` with placeholders for `NEXT_PUBLIC_API_BASE_URL` and required public Cognito settings. Document local

QUALITY
Preserve the existing design. Run lint, type checks and production build. Fix all errors that are possible locally. Provide a
```